MITRC

Modern Institute of Technology & Research Centre, Alwar

Name of Faculty : Poonam Sharma

Subjects & Code : Deep Learning & 7CS4-01

Sem : VIIth

Session : 2025-26 (Odd)

Branch : CSE

Date of submission : 08/09/2025

Unit : II

Lecture No.	Topic Covered	Page No.	Total Pages
1	CNN Architecture and operations	1 – 7	7
2	Convolution layer and pooling layer	8-11	4
3	Forword and Backword propagation	12-17	6
4	Building a deep Neural Network,Training a deep neural network	18-22	5
5	Training a deep neural network,Hyper parameter tuning, Hidden layers	23-30	8
6	Generalization gap-Under fitting Vs over fiting	31-35	5
7	Optimization and Normalization	36-40	5

Faculty
Academics

HOD

Dean 11/9/25

UNIT-2(CNN)

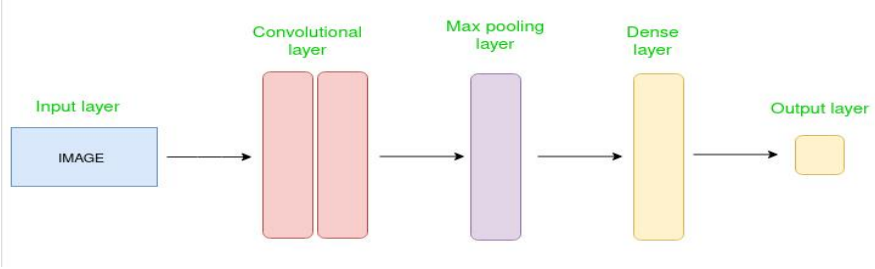
Introduction to Convolution Neural Networks

Lecture No.1

A **Convolutional Neural Network (CNN)** is a type of Deep Learning neural network architecture commonly used in Computer Vision. Computer vision is a field of Artificial Intelligence that enables a computer to understand and interpret the image or visual data. Convolutional Neural Network (CNN) is the extended version of artificial neural networks (ANN) which is predominantly used to extract the feature from the grid-like matrix dataset. For example, visual datasets like images or videos where data patterns play an extensive role.

CNN Architecture

Convolutional Neural Network consists of multiple layers like the input layer, Convolutional layer, Pooling layer, and fully connected layers.

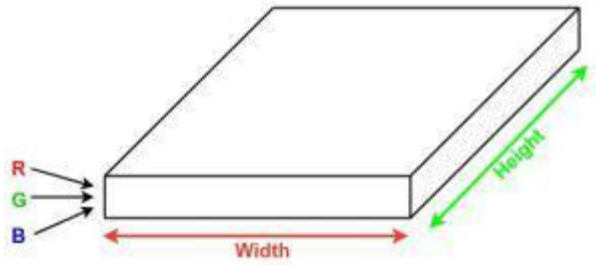


Simple CNN architecture

The Convolutional layer applies filters to the input image to extract features, the Pooling layer downsamples the image to reduce computation, and the fully connected layer makes the final prediction. The network learns the optimal filters through backpropagation and gradient descent.

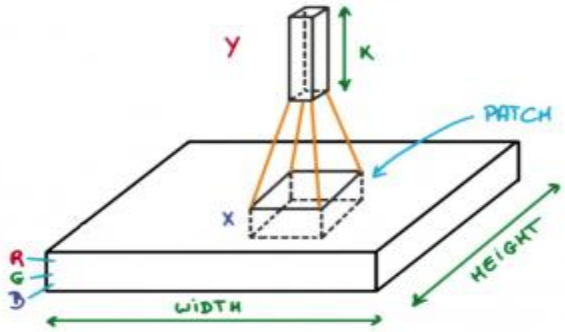
How Convolutional Layers works?

Convolution Neural Networks or covnets are neural networks that share their parameters. Imagine you have an image. It can be represented as a cuboid having its length, width (dimension of the image), and height (i.e the channel as images generally have red, green, and blue channels).



Dimension of the image

Now imagine taking a small patch of this image and running a small neural network, called a filter or kernel on it, with say, K outputs and representing them vertically. Now slide that neural network across the whole image, as a result, we will get another image with different widths, heights, and depths. Instead of just R, G, and B channels now we have more channels but lesser width and height. This operation is called **Convolution**. If the patch size is the same as that of the image it will be a regular neural network. Because of this small patch, we have fewer weights.



Learnable Filters

Now let's talk about a bit of mathematics that is involved in the whole convolution process.

- Convolution layers consist of a set of learnable filters (or kernels) having small widths and heights and the same depth as that of input volume (3 if the input layer is image input).
- For example, if we have to run convolution on an image with dimensions 34x34x3. The possible size of filters can be axax3, where 'a' can be anything like 3, 5, or 7 but smaller as compared to the image dimension.

- During the forward pass, we slide each filter across the whole input volume step by step where each step is called **stride** (which can have a value of 2, 3, or even 4 for high-dimensional images) and compute the dot product between the kernel weights and patch from input volume.
- As we slide our filters we'll get a 2-D output for each filter and we'll stack them together as a result, we'll get output volume having a depth equal to the number of filters. The network will learn all the filters.

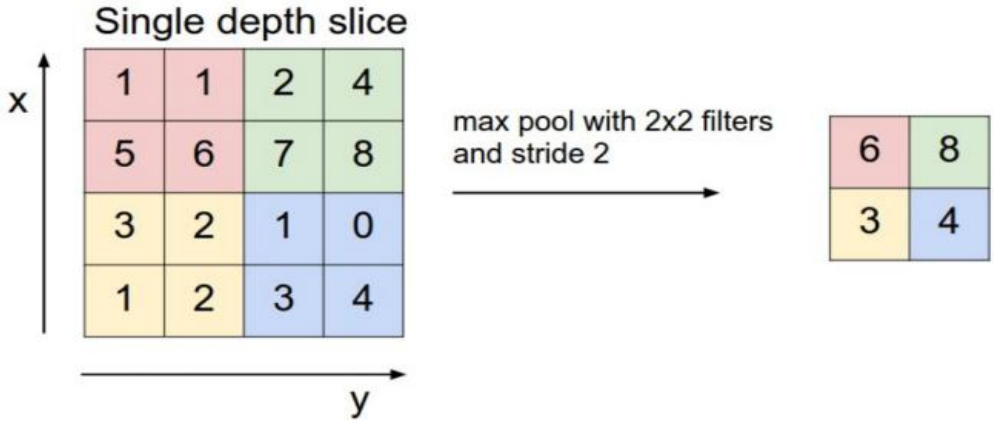
Layers used to build ConvNets

A complete Convolution Neural Networks architecture is also known as covnets. A covnets is a sequence of layers, and every layer transforms one volume to another through a differentiable function**.**

Types of layers

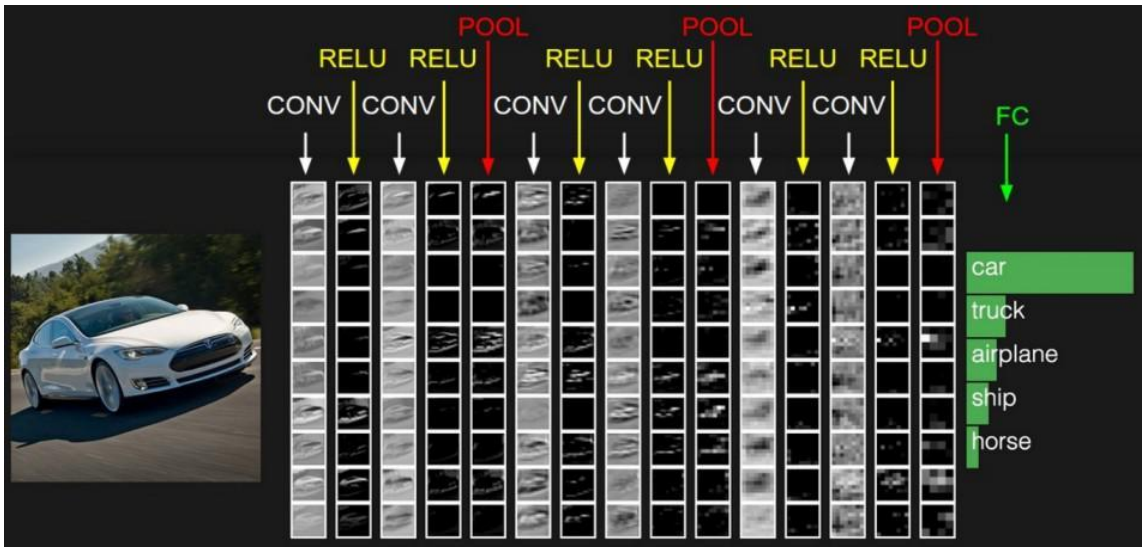
Let's take an example by running a covnets on of image of dimension 32 x 32 x 3.

- **Input Layers:** It's the layer in which we give input to our model. In CNN, Generally, the Input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3.
- **Convolutional Layers:** This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices usually 2×2, 3×3, or 5×5 shape. It slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch. The output of this layer is referred as feature maps. Suppose we use a total of 12 filters for this layer we'll get an output volume of dimension 32 x 32 x 12.
- **Activation Layer:** By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are **RELU**: max(0, x), **Tanh**, **Leaky RELU**, etc. The volume remains unchanged hence output volume will have dimensions 32 x 32 x 12.
- **Pooling layer:** This layer is periodically inserted in the covnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are **max pooling** and **average pooling**. If we use a max pool with 2 x 2 filters and stride 2, the resultant volume will be of dimension 16x16x12.



Max Pooling Process

- **Flattening:** The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.
- **Fully Connected Layers:** It takes the input from the previous layer and computes the final classification or regression task.



Fully connected layer

- **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like sigmoid or softmax which converts the output of each class into the probability score of each class.

References

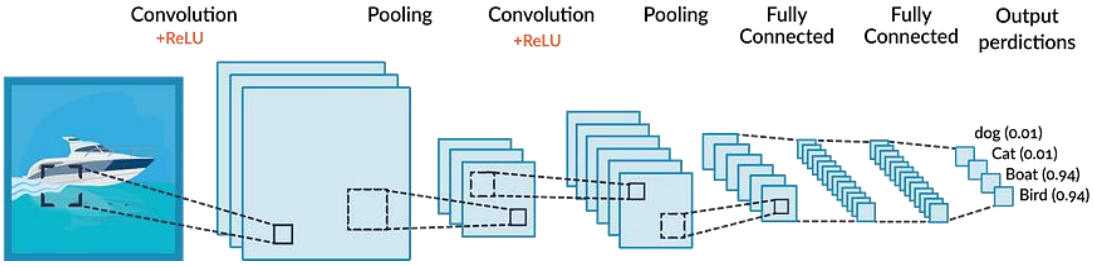
1. https://www.geeksforgeeks.org/introduction-convolution-neural-network/

UNIT-2(CNN)

CNN Operations

Lecture No.2

CNN Structure



CNN Layers

Convolutional networks belong to a class of neural networks that take the image as an input, subjects it to combinations of weights and biases, extracts features and outputs the results. They tend to reduce the dimensions of the input image with the use of a kernel which makes it easier to extract features as compared to a generic Dense neural network. Convolutional networks trace their foundation to **convolution operations** on matrices.

Convnets were inspired by biological processes in that the connectivity pattern between neurons resembles the organization of the animal visual cortex. Individual cortical neurons respond to stimuli only in a restricted region of the visual field known as the receptive field. The receptive fields of different neurons partially overlap such that they cover the entire visual field.

Convolution operations

The name “Convolutional neural network” indicates that the network employs a mathematical operation called Convolution. Convolution is a specialized kind of linear operation. Convnets are simply neural networks that use convolution in place of general matrix multiplication in at least one of their layers.Convolution between two functions in mathematics produces a third function expressing how the shape of one function is modified by other.

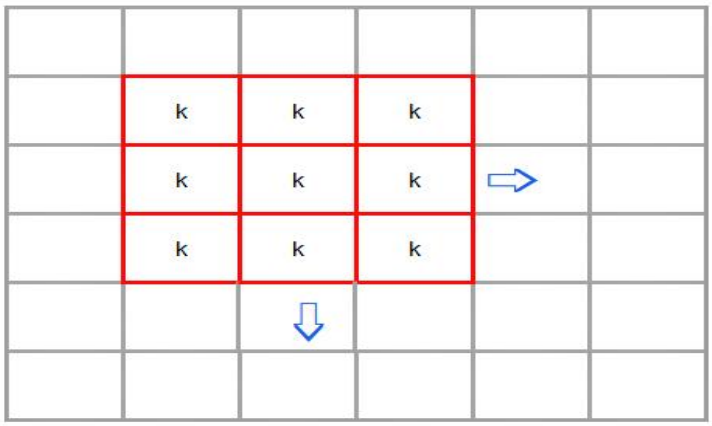
Convolution Kernels

A kernel is a small 2D matrix whose contents are based upon the operations to be performed. A kernel maps on the input image by simple matrix multiplication and addition, the output obtained is of lower dimensions and therefore easier to work with.

Original	Gaussian Blur	Sharpen	Edge Detection
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$
			

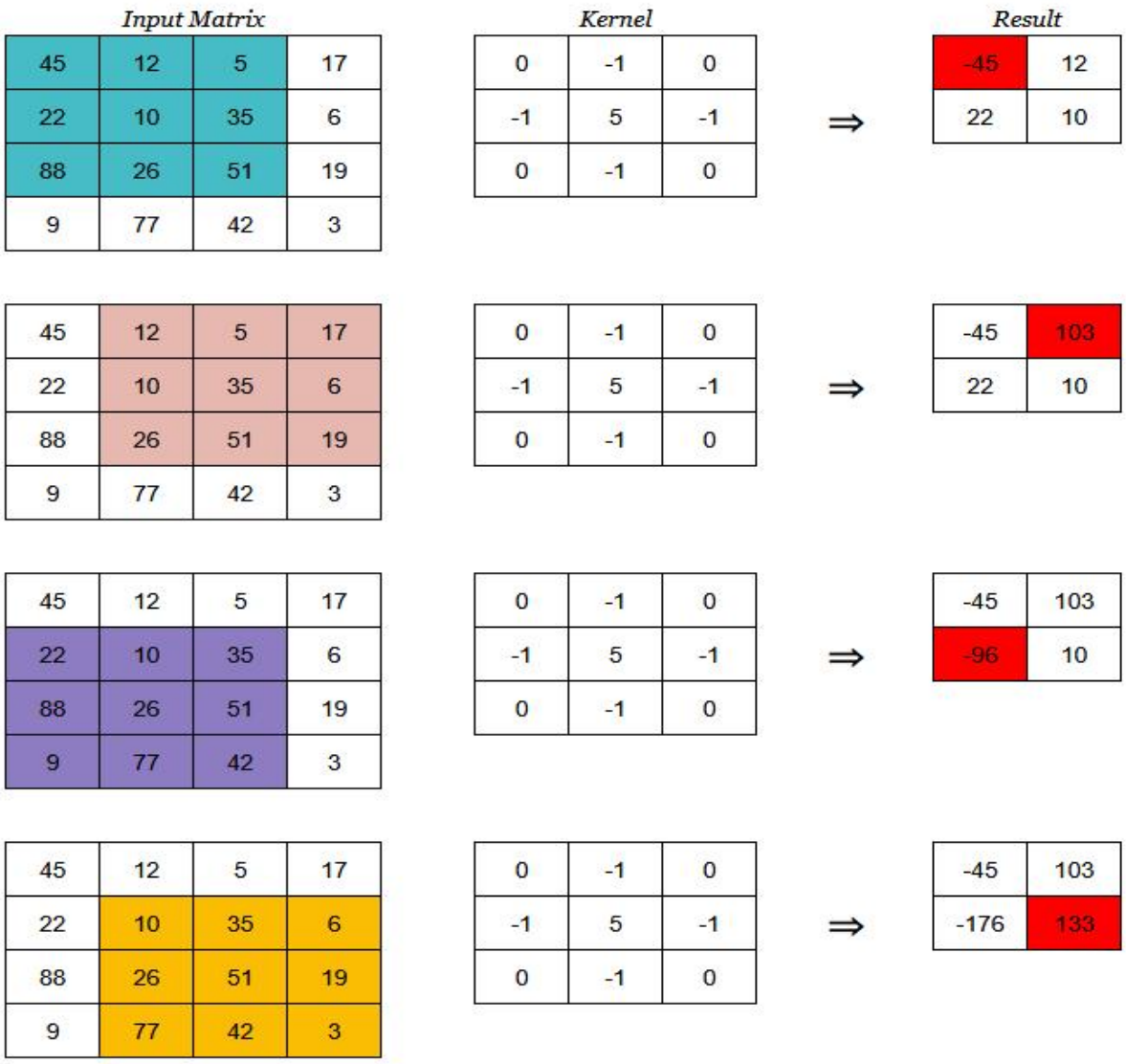
Kernal Map

Above is an example of a kernel for applying Gaussian blur(to smoothen the image before processing), Sharpen image(enhance the depth of edges) and edge detection.The shape of a kernel is heavily dependent on the input shape of the image and architecture of the entire network, mostly the size of kernels is **(MxM)** i.e a square matrix. The movement of a kernel is always from left to right and top to bottom.



Movement of the kernel

Stride defines by what step does to kernel move, for example stride of 1 makes kernel slide by one row/column at a time and stride of 2 moves kernel by 2 rows/columns. Filters are one dimension higher than kernels and can be seen as multiple kernels stacked on each other where every kernel is for a particular channel. Therefore for an RGB image of (32x32) we have a filter of the shape say (5x5x3). Now let's see how a kernel operates on sample matrix.



Here the input matrix has shape 4x4x1 and the kernel is of size 3x3 since the shape of input is larger than the kernel, we are able to implement a sliding window protocol and apply the kernel over entire input. First entry in the convoluted result is calculated as:

*45*0 + 12*(-1) + 5*0 + 22*(-1) + 10*5 + 35*(-1) + 88*0 + 26*(-1) + 51*0 = -45*

Sliding window protocol

1. The kernel gets into position at the top-left corner of the input matrix.
2. Then it starts moving left to right, calculating the dot product and saving it to a new matrix until it has reached the last column.
3. Next, kernel resets its position at first column but now it slides one row to the bottom. Thus following the fashion left-right and top-bottom.
4. Steps 2 and 3 are repeated till the entire input has been processed.

For a 3D input matrix the movement of the kernel will be from front to back, left to right and top to bottom. By now you must have a basic understanding of the Convolution operation which is the essence of a Convolutional Neural Network.

References

1. https://medium.com/analytics-vidhya/understanding-convolution-operations-in-cnn-1914045816d4

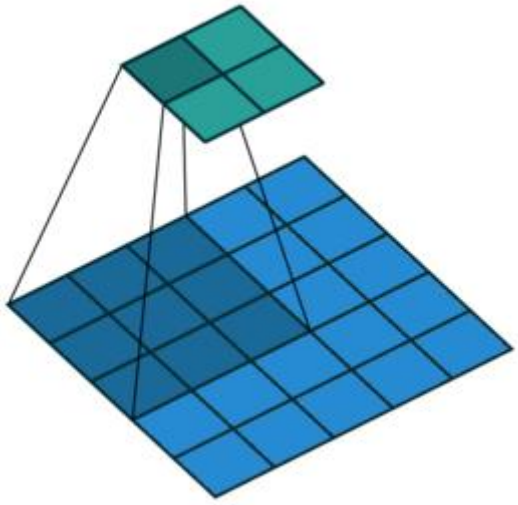
UNIT-2(CNN)

Variants of the Basic Convolution Function

Lecture No.3

In practical implementations of the convolution operation, certain modifications are made which deviate from the discrete convolution formula mentioned above:

- In general a convolution layer consists of application of several different kernels to the input. This allows the extraction of several different features at all locations in the input. This means that in each layer, a single kernel (filter) isn't applied. Multiple kernels (filters), usually a power of 2, are used as different feature detectors.
- The input is generally not real-valued but instead vector valued (e.g. RGB values at each pixel or the feature values computed by the previous layer at each pixel position). Multi-channel convolutions are commutative only if number of output and input channels is the same.
- In order to allow for calculation of features at a **coarser level** *strided convolutions* can be used. The effect of strided convolution is the same as that of a convolution followed by a downsampling stage. This can be used to reduce the representation size.



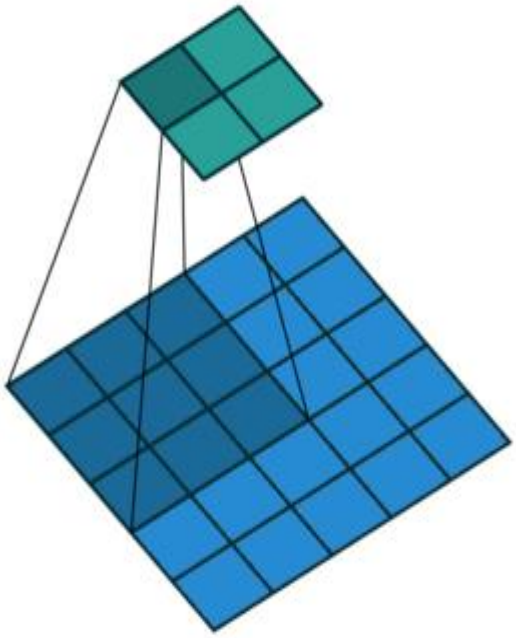
2D convolution 3x3 kernel and stride of 2 units

- Zero padding helps to make output dimensions and kernel size independent. 3 common zero padding strategies are:

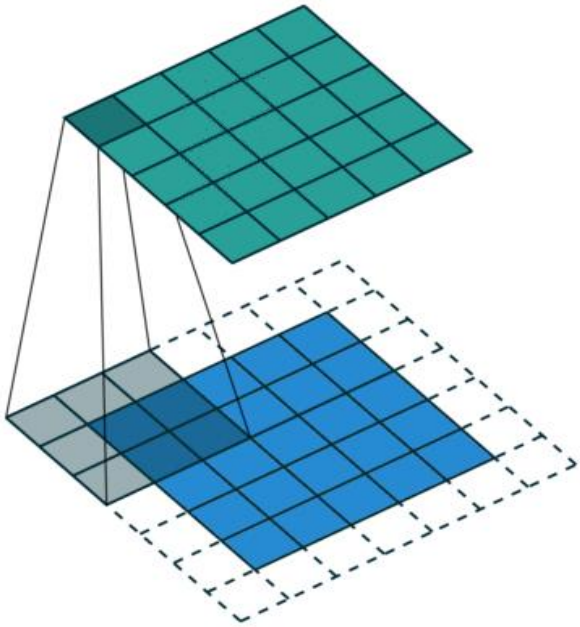
1. **valid**: The output is computed only at places where the entire kernel lies inside the input. Essentially, no zero padding is performed. For a kernel of size **k** in any dimension, the

input shape of **m** in the direction will become **m-k+1** in the output. This shrinkage restricts architecture depth.

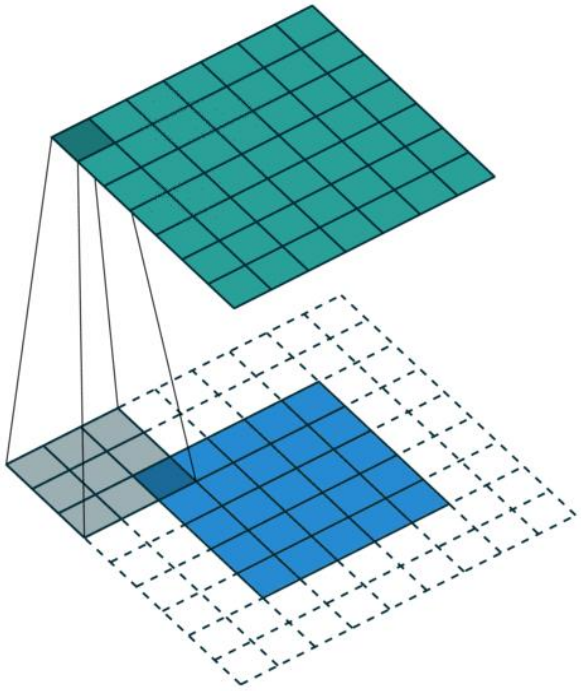
2. **same**: The input is zero padded such that the spatial size of the input and output is same. Essentially, for a dimension where kernle size is **k**, the input is padded by **k-1** zeros in that dimension. Since the number of output units connected to border pixels is less than that for centre pixels, it may under-represent border pixels.
3. **full**: The input is padded by enough zeros such that each input pixel is connected to the same number of output units.In terms of test set accuracy, the optimal padding is somewhere between *same* and *valid.*



valid*(left)*



same*(middle)*



full*(right) padding. The extreme left one is for stride=2.*

- Besides locally-connected layers and tiled convolution, another extension can be to restrict the kernels to operate on certain input channels. One way to implement this is to connect the first **m** input channels to the first **n** output channels, the next **m** input channels to the next **n** output channels and so on. This method decreases the number of parameters in the model without dereasing the number of output units.
- When max pooling operation is applied to locally connected layer or tiled convolution, the model has the ability to become **transformation invariant** because adjacent filters have the freedom to learn a transformed version of the same feature. *This essentially similar to the property leveraged by pooling over channels rather than spatially*.
- **Bias** terms can be used in different ways in the convolution stage. For locally connected layer and tiled convolution, we can use a bias per output unit and kernel respectively. In case of traditional convolution, a single bias term per output channel is used. *If the input size is fixed, a bias per output unit may be used to counter the effect of regional image statistics and smaller activations at the boundary due to zero padding*.

References

1. https://cedar.buffalo.edu/~srihari/CSE676/9.5%20CNN-Variants.pdf

UNIT-2(CNN)

Forward Propagation and Backward Propagation

Lecture No.4

Forward Propagation

Images are fed into the input layer in the form of numbers. These numerical values denote the intensity of pixels in the image. The neurons in the hidden layers apply a few mathematical operations on these values .In order to perform these mathematical operations, there are certain parameter values that are randomly initialized. Post these mathematical operations at the hidden layer, the result is sent to the output layer which generates the final prediction.

Backward Propagation

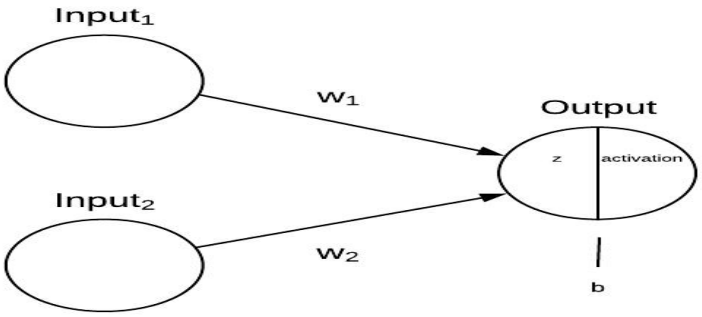
Once the output is generated, the next step is to compare the output with the actual value. Based on the final output, and how close or far this is from the actual value (error), the values of the parameters are updated. The forward propagation process is repeated using the updated parameter values and new outputs are generated.The logical conjunction (AND operator) takes two inputs and returns one output. The function only returns true, if both of its inputs are true. The truth table of it looks like this:

p	q	p & q
1	1	1
0	1	0
0	0	0
1	0	0

Our neural network has 2 inputs, p and q, and one output, the prediction of p & q. Our training set includes 2 examples out of the 4 possible examples:

$$x_1 = [1, \ 1]$$
$$x_2 = [0, \ 1]$$

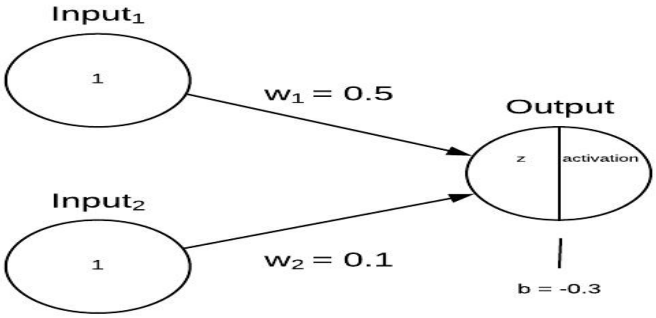
For the first training example, the desired output of the neural network is 1, and for the second training example, it is 0. Our neural network has 2 neurons in the input layer and 1 neuron on the output layer. The structure looks like this:



Forward Propagation

Forward propagation

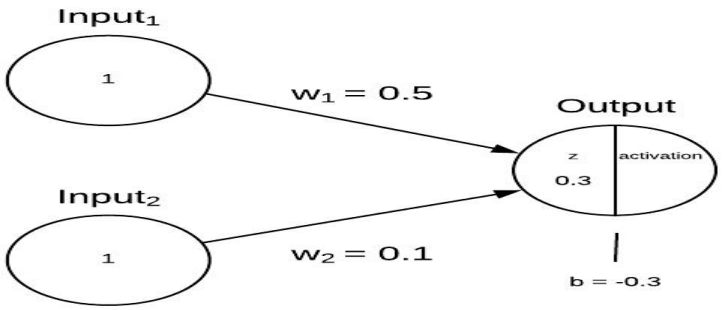
In the forward propagation, we check what the neural network predicts for the first training example with initial weights and bias. First, we initialize the weights and bias randomly:



Then we calculate z, the weighted sum of activation and bias:

$$z = Input_1 \times w_1 + Input_2 \times w_2 + b$$

$$z = 1 \times 0.5 + 1 \times 0.1 - 0.3 = 0.3$$



After we have z, we can apply the activation function to it:

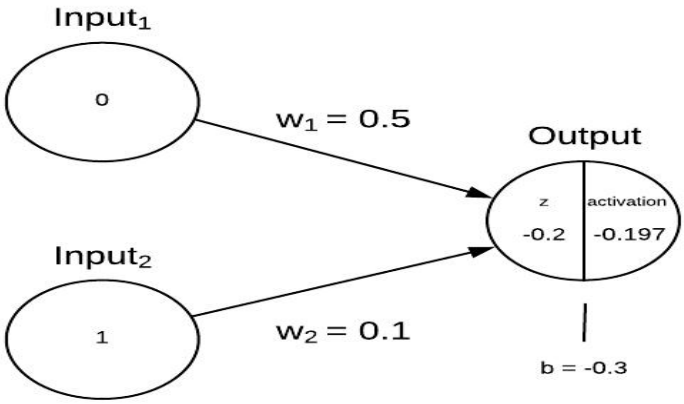
$$Output = \sigma(z)$$

σ is the activation function. The most common activation functions are *relu, sigmoid* and *tanh*. In this example, we are going to use *tanh*.

$$Output = tanh(0.3) = 0.291$$

For the first training example, our neural network predicted the outcome 0.291. Our desired outcome is 1. The neural network can improve with the learning process of backward propagation. Before we continue with the backward propagation, let's calculate the prediction for the second training example. Here are the results:

$$x_2 = [0, 1]$$



Backward propagation

We can define a cost function that measures how good our neural network performs. For an input, x, and desired output, y, we can calculate the cost of a specific training example as the square of the difference between the network's output and the desired output, that is,

$$C_k = (Output - y)^2$$

where *k* stands for the training example and the output is assumed to be the activation of the output neuron, and *y* is the actual desired output. For our training examples, the costs are the following:

$$C_1 = (0.291 - 1)^2 = 0.502$$

$$C_2 = (-0.197 - 0)^2 = 0.038$$

The total cost of a training set is the average of the individual cost functions of the data in the training set:

$$C = \frac{1}{N} \sum C_k$$

where *N* stands for the number of training examples. In our training set it looks like this:

$$C = \frac{0.502 + 0.038}{2} = 0.27$$

We want to improve the performance of the neural network on the training examples, so that we can change the weights and bias, and hopefully, lower the total cost. We want to know how much the specific weights and bias affect the total cost, so we need to calculate the partial derivatives of the total cost with respect to the weights and bias. To do this, we can apply the chain rule:

$$\frac{\partial C_k}{\partial w_1} = \frac{\partial C_k}{\partial Output} \frac{\partial Output}{\partial z} \frac{\partial z}{\partial w_1}$$

$$\frac{\partial C_k}{\partial w_2} = \frac{\partial C_k}{\partial Output} \frac{\partial Output}{\partial z} \frac{\partial z}{\partial w_2}$$

$$\frac{\partial C_k}{\partial b} = \frac{\partial C_k}{\partial Output} \frac{\partial Output}{\partial z} \frac{\partial z}{\partial b}$$

After simplification, the parts look like this:

$$\frac{\partial C_k}{\partial Output} = 2(Output - y)$$

$$\frac{\partial Output}{\partial z} = \sigma'(z)$$

$$\frac{\partial z}{\partial w_1} = Input_1$$

$$\frac{\partial z}{\partial w_2} = Input_2$$

$$\frac{\partial z}{\partial b} = 1$$

The calculation of the partial derivatives for the first training example:

$$\frac{\partial C_1}{\partial Output} = 2(0.291 - 1) = -1.418$$

$$\frac{\partial Output}{\partial z} = \frac{1}{\cosh^2 0.3} = 0.915$$

$$\frac{\partial z}{\partial w_1} = 1$$

$$\frac{\partial z}{\partial w_2} = 1$$

$$\frac{\partial z}{\partial b} = 1$$

$$\frac{\partial C_1}{\partial w_1} = (-1.418) \times 0.915 \times 1 = -1.297$$

$$\frac{\partial C_1}{\partial w_2} = (-1.418) \times 0.915 \times 1 = -1.297$$

$$\frac{\partial C_1}{\partial b} = (-1.418) \times 0.915 \times 1 = -1.297$$

The partial derivatives for the second training example:

$$\frac{\partial C_2}{\partial w_1} = 0$$

$$\frac{\partial C_2}{\partial w_2} = -0.379$$

$$\frac{\partial C_2}{\partial b} = -0.379$$

Now, we calculate the partial derivatives with respect to the total cost. Consider the first weight (w1). The partial derivative of the total cost with respect to w1 is the average of all the partial derivatives of the individual cost functions with respect to w1:

$$\frac{\partial C}{\partial w_1} = \frac{1}{N} \sum_{k=0}^{N-1} \frac{\partial C_k}{\partial w_1}$$

In our training example, it is:

$$\frac{\partial C}{\partial w_1} = \frac{(-1.297) + 0}{2} = -0.648$$

We do the same calculation for the other weight and bias:

$$\frac{\partial C}{\partial w_2} = \frac{(-1.297) + (-0.379)}{2} = -0.838$$

$$\frac{\partial C}{\partial b} = \frac{(-1.297) + (-0.379)}{2} = -0.838$$

Then we update the weights and bias: we multiply the partial derivatives with some learning rate and subtract the results from the weights and bias. Let's use the learning rate $\alpha = 0.6$

$$w_1^+ = w_1 - (\alpha \times \frac{\partial C}{\partial w_1})$$

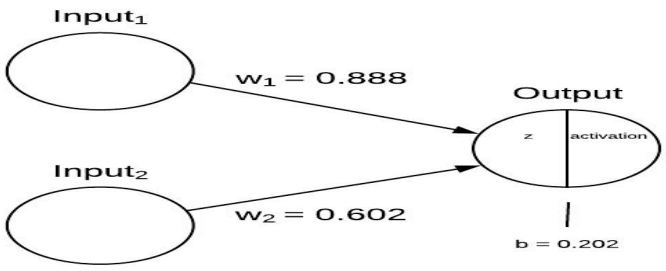
$$w_1^+ = 0.5 - (0.6 \times (-0.648)) = 0.888$$

Repeating this calculation with the other weight and the bias:

$$w_2^+ = 0.602$$

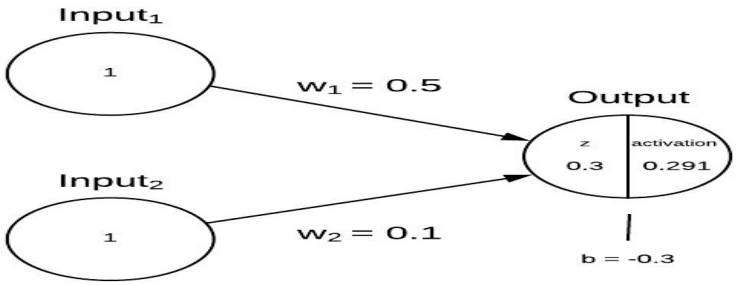
$$b^+ = 0.202$$

After updating the weights and bias, our neural network looks like this:



Iteration of backward propagation

This is the end of the first iteration of the backward propagation. We could continue with the forward propagation, calculate the cost, and then go back to the backward propagation again.



References

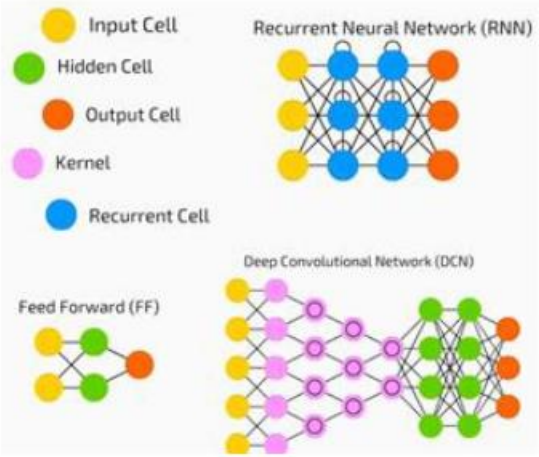
1. https://www.geeksforgeeks.org/backpropagation-in-neural-network/

UNIT-2(CNN)

Building a Deep Neural Network- Improving Deep Neural Networks- Training a deep neural network

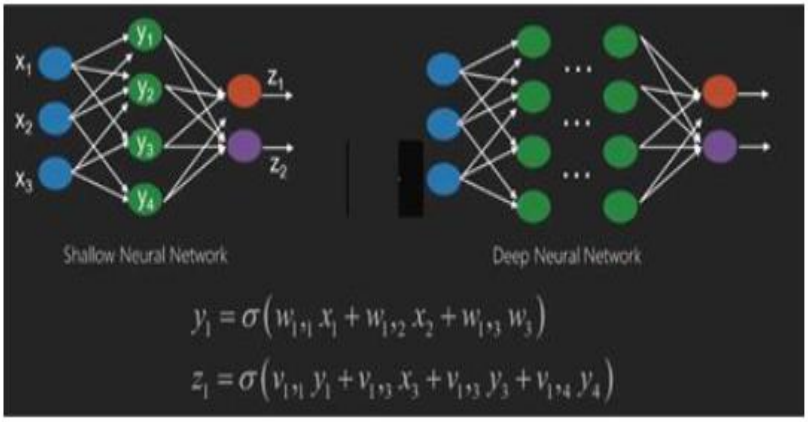
Lecture No.5

A deep neural network (DNN) is an ANN with multiple hidden layers between the input and output layers. Similar to shallow ANNs, DNNs can model complex non-linear relationships.
The main purpose of a neural network is to receive a set of inputs, perform progressively complex calculations on them, and give output to solve real world problems like classification. We restrict ourselves to feed forward neural networks. We have an input, an output, and a flow of sequential data in a deep network.

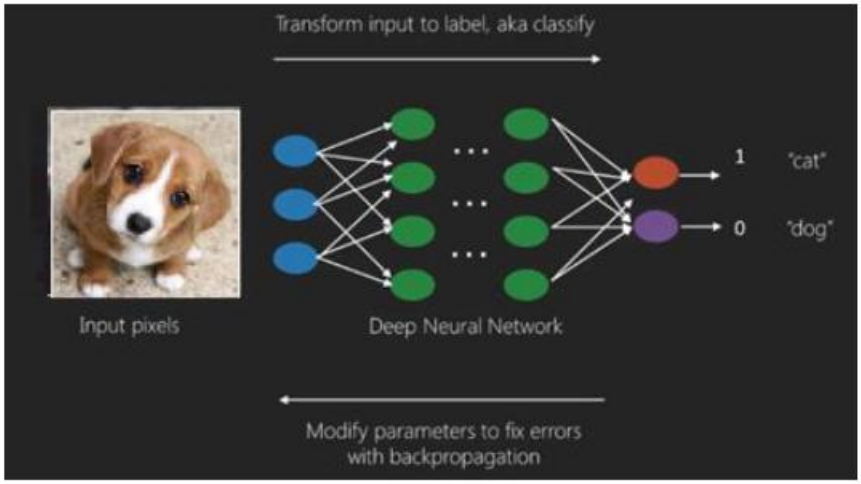


Neural networks are widely used in supervised learning and reinforcement learning problems. These networks are based on a set of layers connected to each other. In deep learning, the number of hidden layers, mostly non-linear, can be large; say about 1000 layers. DL models produce much better results than normal ML networks.We mostly use the gradient descent method for optimizing the network and minimizing the loss function. We can use the **Imagenet**, a repository of millions of digital images to classify a dataset into categories like cats and dogs. DL nets are increasingly used for dynamic images apart from static ones and for time series and text analysis.

Training the data sets forms an important part of Deep Learning models. In addition, Backpropagation is the main algorithm in training DL models. DL deals with training large neural networks with complex input output transformations.One example of DL is the mapping of a photo to the name of the person(s) in photo as they do on social networks and describing a picture with a phrase is another recent application of DL.



Neural networks are functions that have inputs like x1,x2,x3…that are transformed to outputs like z1,z2,z3 and so on in two (shallow networks) or several intermediate operations also called layers (deep networks).The weights and biases change from layer to layer. ‘w’ and ‘v’ are the weights or synapses of layers of the neural networks. The best use case of deep learning is the supervised learning problem. Here, we have large set of data inputs with a desired set of outputs.



Here we apply back propagation algorithm to get correct output prediction. The most basic data set of deep learning is the MNIST, a dataset of handwritten digits.We can train deep a Convolutional Neural Network with Keras to classify images of handwritten digits from this dataset. The firing or activation of a neural net classifier produces a score. For example, to classify patients as sick and healthy, we consider parameters such as height, weight and body temperature, blood pressure etc. A high score means patient is sick and a low score means he is healthy.

Each node in output and hidden layers has its own classifiers. The input layer takes inputs and passes on its scores to the next hidden layer for further activation and this goes on till

the output is reached.This progress from input to output from left to right in the forward direction is called **forward propagation.**Credit assignment path (CAP) in a neural network is the series of transformations starting from the input to the output. CAPs elaborate probable causal connections between the input and the output.

CAP depth for a given feed forward neural network or the CAP depth is the number of hidden layers plus one as the output layer is included. For recurrent neural networks, where a signal may propagate through a layer several times, the CAP depth can be potentially limitless.

Deep Nets and Shallow Nets

There is no clear threshold of depth that divides shallow learning from deep learning; but it is mostly agreed that for deep learning which has multiple non-linear layers, CAP must be greater than two.Basic node in a neural net is a perception mimicking a neuron in a biological neural network. Then we have multi-layered Perception or MLP. Each set of inputs is modified by a set of weights and biases; each edge has a unique weight and each node has a unique bias.

The prediction **accuracy** of a neural net depends on its **weights and biases.** The process of improving the accuracy of neural network is called **training.** The output from a forward prop net is compared to that value which is known to be correct.The **cost function or the loss function** is the difference between the generated output and the actual output.

The point of training is to make the cost of training as small as possible across millions of training examples. To do this, the network tweaks the weights and biases until the prediction matches the correct output.

Once trained well, a neural net has the potential to make an accurate prediction every time. When the pattern gets complex and you want your computer to recognise them, you have to go for neural networks. In such complex pattern scenarios, neural network outperforms all other competing algorithms. There are now GPUs that can train them faster than ever before. Deep neural networks are already revolutionizing the field of AI.

Computers have proved to be good at performing repetitive calculations and following detailed instructions but have been not so good at recognising complex patterns.

If there is the problem of recognition of simple patterns, a support vector machine (svm) or a logistic regression classifier can do the job well, but as the complexity of patternincreases, there is no way but to go for deep neural networks. Therefore, for complex patterns like a human face, shallow neural networks fail and have no alternative but to go for deep neural networks with more layers. The deep nets are able to do their job by breaking down the complex patterns into simpler ones. For example, human face; a deep net would use edges to detect parts like lips, nose, eyes, ears and so on and then re-combine these together to form a human face.

The accuracy of correct prediction has become so accurate that recently at a Google Pattern Recognition Challenge, a deep net beat a human. This idea of a web of layered perceptrons has been around for some time; in this area, deep nets mimic the human brain. But one downside to this is that they take long time to train, a hardware constraint.However recent high performance GPUs have been able to train such deep nets under a week; while fast cpus could have taken weeks or perhaps months to do the same.

References

1. https://medium.com/@douglasreiser/building-a-deep-neural-network-from-scratch-in-typescript-9028903c15f1

UNIT-2(CNN)

Hyper-parameter Tuning

Lecture No.6

Hyper-parameter tuning is an essential part of controlling the behavior of a machine learning model. If we don't correctly tune our hyper-parameters, our estimated model parameters produce suboptimal results, as they don't minimize the loss function. This means our model makes more errors.

What are hyper-parameters?

In machine learning, we need to differentiate between parameters and hyper-parameters. A learning algorithm learns or estimates model parameters for the given data set, then continues updating these values as it continues to learn. After learning is complete, these parameters become part of the model. For example, each weight and bias in a neural network is a parameter.Hyper-parameters, on the other hand, are specific to the algorithm itself, so we can't calculate their values from the data. We use hyper-parameters to calculate the model parameters. Different hyper-parameter values produce different model parameter values for a given data set.

Hyper-parameter tuning consists of finding a set of optimal hyper-parameter values for a learning algorithm while applying this optimized algorithm to any data set. That combination of hyper-parameters maximizes the model's performance, minimizing a predefined loss function to produce better results with fewer errors. Note that the learning algorithm optimizes the loss based on the input data and tries to find an optimal solution within the given setting. However, hyper-parameters describe this setting exactly.
For instance, if we work on natural language processing (NLP) models, we probably use neural networks, support-vector machines (SVMs), Bayesian networks, and Extreme Gradient Boosting (XGB) for tuning parameters.

Important hyperparameters that require tuning in neural networks:
Number of hidden layers: It's a trade-off between keeping our neural network as simple as possible (fast and generalized) and classifying our input data correctly. We can start with values of four to six and check our data's prediction accuracy when we increase or decrease this hyperparameter.

1. Number of nodes/neurons per layer: More isn't always better when determining how many neurons to use per layer. Increasing neuron count can help, up to a point. But layers that are too wide may memorize the training dataset, causing the network to be less accurate on new data.

2. Learning rate: Model parameters are adjusted iteratively — and the learning rate controls the size of the adjustment at each step. The lower the learning rate, the lower the changes to parameter estimates are. This means that it takes a longer time (and more data) to fit the model — but it also means that it is more likely that we actually find the minimum loss.

3. Momentum: Momentum helps us avoid falling into local minima by resisting rapid changes to parameter values. It encourages parameters to keep changing in the direction they were already changing, which helps prevent zig-zagging on every iteration. Aim to start with low momentum values and adjust upward as needed.

Essential hyperparameters for tuning SVMs:

1. C: A trade-off between a smooth decision boundary (more generic) and a neat decision boundary (more accurate for the training data). A low value may cause the model to incorrectly classify some training data, while a high value may cause the model to incur overfitting. Overfitting creates an analysis too specific for the current data set and possibly unfit for future data and unreliable for future observations.

2. Gamma: The inverse of the influence radius of data samples we selected as support vectors. High values indicate the small radius of influence and small decision boundaries that do not consider relatively close data samples. These high values cause overfitting. Low values indicate the significant effect of distant data samples, so the model can't capture the correct decision boundaries from the data set.

Important hyperparameters that need tuning for XGBoost:

1. max_depth and min_child_weight:

This controls the tree architecture. **max_depth** defines the maximum number of nodes from the root to the farthest leaf (the default number is **min_child_weight i**s the minimum weight required to create a new node in the tree.

2. learning_rate:

This determines the amount of correction at each step, given that each boosting round corrects the previous round's errors. learning_rate takes values from 0 to 1, and the default value is 0.3.

3. n_estimators:

This defines the number of trees in the ensemble. The default value is 100. Note that if we were using vanilla XGBoost instead of scikit-learn, we'd use **num_boost_rounds instead of n_estimators**.

4. colsample_bytree and subsample:

This controls the data set samples that each round uses. These hyperparameters are helpful to avoid overfitting. **subsample** is the fraction of samples used, with a value from 0 to 1 and a default value of 1.

colsample_bytree defines the fraction of columns (features) and takes numbers from 0 to 1, with a default value of 1.In these examples, hyperparameters tending toward one extreme or the other can negatively affect our model's ability to make predictions. The trick is to find just the right value for each hyperparameter, so our model performs well and produces the best possible results.

Methods for tuning hyperparameters

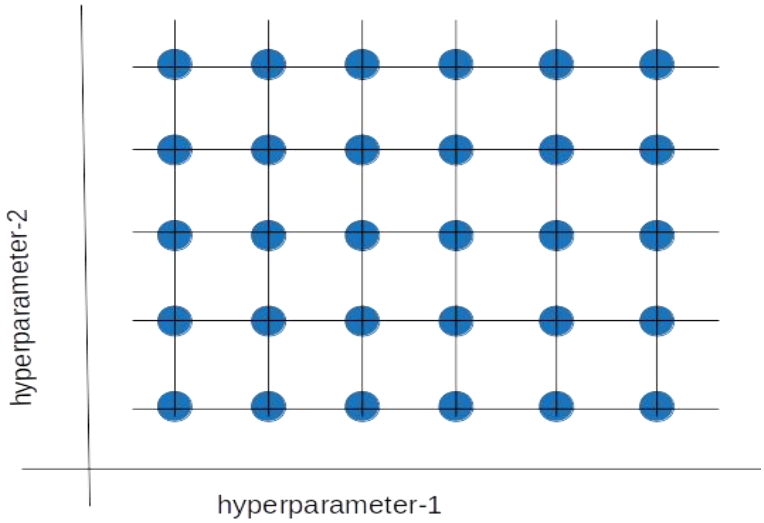
Now that we understand what hyperparameters are and the importance of tuning them, we need to know how to choose their optimal values. We can find these optimal hyperparameter values using manual or automated methods.

When tuning hyperparameters manually, we typically start using the default recommended values or rules of thumb, then search through a range of values using trial-and-error. But manual tuning is a tedious and time-consuming approach. It isn't practical when there are many hyperparameters with a wide range.

Automated hyperparameter tuning methods use an algorithm to search for the optimal values. Some of today's most popular automated methods are grid search, random search, and Bayesian optimization. Let's explore these methods in detail.

1. Grid search

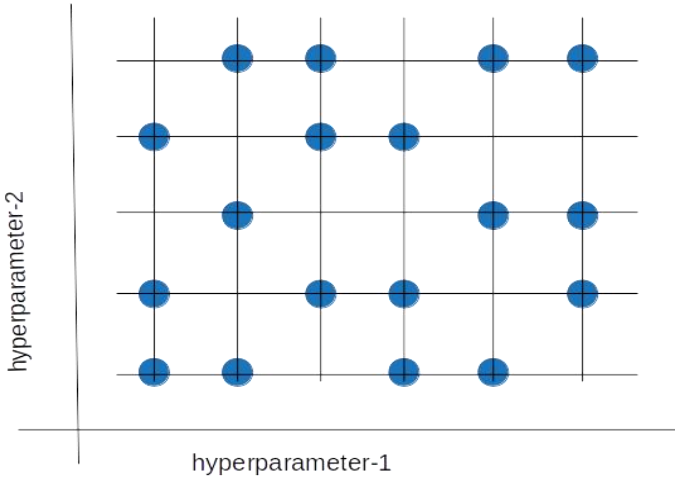
Grid search is a sort of "brute force" hyperparameter tuning method. We create a grid of possible discrete hyperparameter values then fit the model with every possible combination. We record the model performance for each set then select the combination that has produced the best performance.



Grid search is a hyperparameter tuning method in which we create a grid of possible discrete hyperparameter values, then fit the model with every possible combination. Grid search is an exhaustive algorithm that can find the best combination of hyperparameters. However, the drawback is that it's slow. Fitting the model with every possible combination usually requires a high computation capacity and significant time, which may not be available.

2. Random search

The random search method (as its name implies) chooses values randomly rather than using a predefined set of values like the grid search method. Random search tries a random combination of hyper-parameters in each iteration and records the model performance. After several iterations, it returns the mix that produced the best result.



Random search tries a random combination of hyperparameters in each iteration and records the model performance. After several iterations, it returns the mix that produced the best result.

Random search is appropriate when we have several hyperparameters with relatively large search domains. We can make discrete ranges (for instance, [5-100] in steps of 5) and still get a reasonably good set of combinations.
The benefit is that random search typically requires less time than grid search to return a comparable result. It also ensures we don't end up with a model that's biased toward value sets arbitrarily chosen by users. Its drawback is that the result may not be the best possible hyperparameter combination.

3. Bayesian optimization

Grid search and random search are relatively inefficient because they often evaluate many unsuitable hyperparameter combinations. They don't take into account the previous iterations' results when choosing the next hyperparameters.

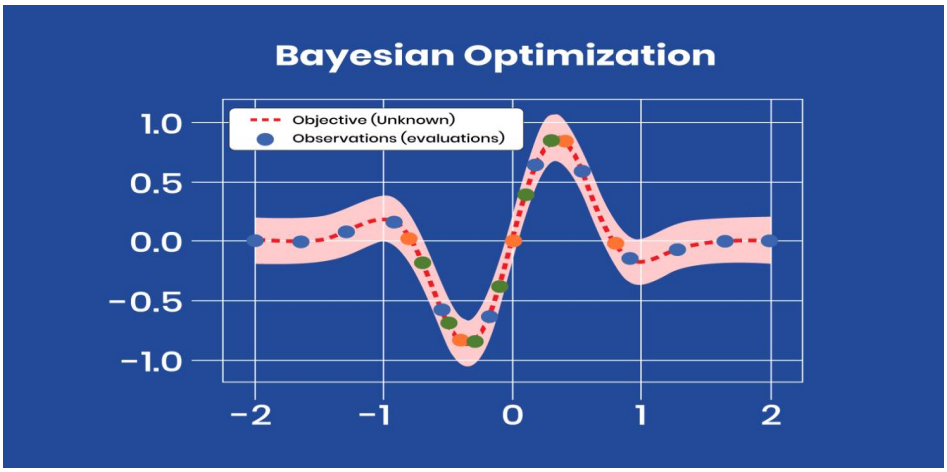
The Bayesian optimization method takes a different approach. This method treats the search for the optimal hyperparameters as an optimization problem. When choosing the next hyperparameter combination, this method considers the previous evaluation results. It then applies a probabilistic function to select the combination that will probably yield the best results. This method discovers a fairly good hyperparameter combination in relatively few iterations.

Data scientists choose a probabilistic model when the objective function is unknown. That is, there is no analytical expression to maximize or minimize. The data scientists apply the learning algorithm to a data set, use the algorithm's results to define the objective function, and take the various hyperparameter combinations as the input domain.
The probabilistic model is based on past evaluation results. It estimates the probability of a hyperparameter combination's objective function result: P(result|hyperparameters).

This probabilistic model is a "surrogate" of the objective function. The objective function can be, for instance, the root-mean-square error (RMSE). We calculate the objective function using the training data with the hyperparameter combination. We try to optimize it (maximize or minimize, depending on the objective function selected).

Applying the probabilistic model to the hyperparameters is computationally inexpensive compared to the objective function, so this method typically updates and improves the surrogate probability model every time the objective function runs. Better hyperparameter predictions decrease the number of objective function evaluations we need to achieve a good result.

Gaussian processes, random forest regression, and tree-structured Parzen estimators (TPE) are surrogate model examples.The figure below shows how a surrogate function finds the minimum of the “objective” function, where the “objective” function is unknown.



After a few iterations, with the evaluations (observations) obtained from the “objective” function, the surrogate model finds the minimum at the coordinates x=-0.35 and y=-0.8. Note that there are several evaluations close to that point, corresponding to the latest iterations, as the model has almost found the minimum and produces similar values (different color points correspond to different iterations).

The Bayesian optimization model is complex to implement. Fortunately, we can use off-the-shelf libraries like **Ray Tune** to simplify the process. It’s worthwhile to use this type of model because it finds an adequate hyperparameter combination in relatively few iterations.

Bayesian optimization is helpful when the objective function is costly in computing resources and time. A drawback compared to grid search or random search is that we must compute Bayesian optimization sequentially (where the next iteration depends on the previous one), so it doesn’t allow distributed processing. So, Bayesian optimization takes longer yet uses fewer computational resources.

References

1. https://www.geeksforgeeks.org/hyperparameter-tuning/

UNIT-2(CNN)

Hidden Layer

Lecture No.7

In the context of AI, a *hidden layer* refers to the intermediate layer of nodes or neurons in a neural network that processes input data and produces the desired output. The term "hidden" signifies that these layers are not directly observable from the system's input or output. The presence of multiple hidden layers allows neural networks to learn complex patterns and relationships within the data, enabling them to make accurate predictions or classifications.

This concept forms the foundation for deep learning, enhancing the network's capacity to represent intricate features and perform sophisticated tasks. The versatility of hidden layers facilitates complex computations and information processing, driving the advancements in AI and machine learning.

How hidden layer works?

The mechanism of hidden layers revolves around their capacity to perform nonlinear transformations on the input data. Each node in a hidden layer applies a transformation to the input data, applying weights and biases to compute an output that is passed to the subsequent layer. As data passes through the hidden layers, it undergoes a series of transformations, progressively capturing higher-level abstractions and patterns.

The dynamic nature of hidden layer computation enables neural networks to learn complex representations of the input data, facilitating the extraction of features that are critical for accurate decision-making. This iterative process of feature extraction and transformation within hidden layers forms the cornerstone of the learning capabilities exhibited by AI systems.

Pros & cons of hidden layers

The integration of hidden layers in AI systems offers a spectrum of benefits and advantages, while also presenting certain limitations and challenges. Understanding the pros and cons of hidden layers is integral to effectively harnessing their capabilities within neural networks:

Pros:

- **Enhanced Learning Capacity:** Hidden layers empower AI systems to discern intricate patterns and relationships within the data, leading to improved learning capabilities.
- **Feature Extraction:** The iterative processing within hidden layers facilitates the extraction of high-level features from raw data, enhancing model accuracy.

- **Complex Problem Solving:** AI models equipped with hidden layers exhibit enhanced capabilities in solving complex problems and making reliable predictions.
- **Adaptability:** Hidden layers enable neural networks to adapt to diverse data distributions and learn from variable inputs.

Cons:

- **Computational Complexity:** The presence of multiple hidden layers can amplify the computational complexity of neural network training and inference.
- **Overfitting:** In certain instances, the presence of hidden layers may lead to overfitting, where the model performs exceptionally well on training data but falters on unseen data.
- **Hyperparameter Sensitivity:** Configuring the number of hidden layers and their respective parameters requires careful tuning, leading to potential challenges in model optimization and performance.

References

1. https://en.wikipedia.org/wiki/Hidden_layer

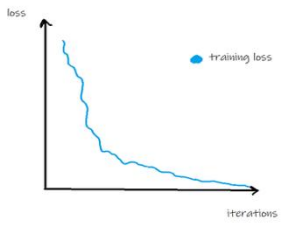
UNIT-2(CNN)
Generalization Gap
Lecture No.8

The ultimate goal of machine learning is to find statistical patterns in a training set that generalize to data outside the training set. Take the following simple NLP problem: Say you want to predict a word in a sequence given its preceding words.

For example, the sequence "the cat ___" may be followed by *sleeps*, *enjoys,* or *wants*. Your training sequences probably include many such noun-verb combinations, others might be"she *has*", "an expert *says*", "it *seems*", and so on. Note that all these verbs have a third person -*s* because they refer to the subject of the sequence. A good language model can indeed pick up this pattern and make correct predictions even on sequences it was not trained on. Given as input "the plane ___", for instance, we would expect the model to predict *departs* with a higher probability than *depart*, and *leaves* with a higher probability than *leave*. In that case, the model has learned a pattern that is more generally true. Thumbs up. Yet the model does not understand the meaning of words, not to mention language in general. If we, for instance, provide "theee plaaaane ___" as input to the model, it will totally screw up.

Is it learning at all?

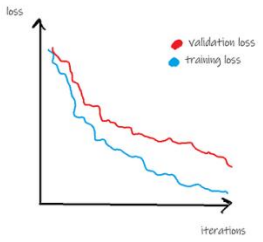
It's usually a good idea to plot the learning process. First, take a look at the training loss. It should approach zero, or at least a reasonably low value (see graphic below). If not, your model is probably either not a good fit, is not flexible enough, or — and that is often forgotten — the output is not a function of the input data so there is no relationship to learn. Think about whether a sentence is really a function of words, for instance. It may not be true unconditionally, but can still be useful in some ways.



Training loss should decrease with the number of training iterations (but not necessarily to 0).

What is generalization?

If training loss in fact does decrease as expected, it doesn't automatically mean that whatever the model has learned is also useful. This is where the validation loss comes into play. Things look good if the validation loss decreases alongside the training loss. In that case, the learned patterns seem to generalize to the unseen validation data. The validation loss will typically be higher than the training loss, however, since not all patterns generalize, as you can see in the following graphic.



If validation loss decreases as well, the learned patterns seem to generalize.

UNIT-2(CNN)

Under–fitting Vs Over-fitting

Lecture No.9

Underfitting in Machine Learning

A statistical model or a machine learning algorithm is said to have underfitting when a model is too simple to capture data complexities. It represents the inability of the model to learn the training data effectively result in poor performance both on the training and testing data. In simple terms, an underfit model's are inaccurate, especially when applied to new, unseen examples. It mainly happens when we uses very simple model with overly simplified assumptions. To address underfitting problem of the model, we need to use more complex models, with enhanced feature representation, and less regularization.

Reasons for Underfitting

1. The model is too simple, So it may be not capable to represent the complexities in the data.
2. The input features which is used to train the model is not the adequate representations of underlying factors influencing the target variable.
3. The size of the training dataset used is not enough.
4. Excessive regularization are used to prevent the overfitting, which constraint the model to capture the data well.
5. Features are not scaled.

Techniques to Reduce Underfitting

1. Increase model complexity.
2. Increase the number of features, performing feature engineering.
3. Remove noise from the data.
4. Increase the number of epochs or increase the duration of training to get better results.

Overfitting in Machine Learning

A statistical model is said to be overfitted when the model does not make accurate predictions on testing data. When a model gets trained with so much data, it starts learning from the noise and inaccurate data entries in our data set. And when testing with

test data results in High variance. Then the model does not categorize the data correctly, because of too many details and noise.

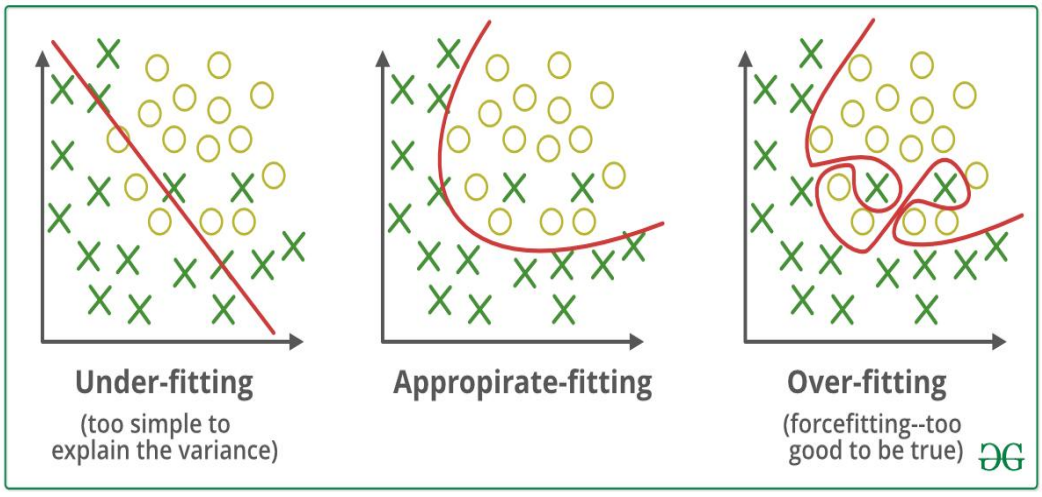
The causes of overfitting are the non-parametric and non-linear methods because these types of machine learning algorithms have more freedom in building the model based on the dataset and therefore they can really build unrealistic models. A solution to avoid overfitting is using a linear algorithm if we have linear data or using the parameters like the maximal depth if we are using decision trees.
In a nutshell, Overfitting is a problem where the evaluation of machine learning algorithms on training data is different from unseen data.

Reasons for Overfitting:

1. High variance and low bias.
2. The model is too complex.
3. The size of the training data.

Techniques to Reduce Overfitting

1. Increase training data.
2. Reduce model complexity.
3. Early stopping during the training phase (have an eye over the loss over the training period as soon as loss begins to increase stop training).
4. Ridge Regularization and Lasso Regularization.
5. Use dropout for neural networks to tackle overfitting.



Difference between the Under–fitting and Over-fitting:

Overfitting	Underfitting
Model is too complex	Model is not complex enough
Accurate for training set	Not accurate for training set
Not accurate for validation set	Not accurate for validation set
Need to reduce complexity	Need to increase complexity
Reduce number of features	Increase number of features
Apply regularization	Reduce regularization
Reduce training	Increase training
Add training examples	Add training examples

References

1. https://research.google/blog/predicting-the-generalization-gap-in-deep-neural- networks/

UNIT-2(CNN)

Optimization in Deep learning

Lecture No.10

Although optimization may help deep learning by lowering the loss function, the aims of optimization and deep learning are fundamentally different. The former is more focused on minimizing an objective, whereas the latter is more concerned with finding a good model given a finite quantity of data. Training error and generalization error, for example, vary in that the optimization algorithm's objective function is usually a loss function based on the training dataset, and the purpose of optimization is to minimize training error. Deep learning (or, to put it another way, statistical inference) aims to decrease generalization error. In order to achieve the latter, we must be aware of overfitting as well as use the optimization procedure to lower the training error.

Gradient Descent Deep Learning Optimizer-

Gradient Descent is the most common optimizer in the class. Calculus is used in this optimization process to make consistent changes to the parameters and reach the local minimum. Before you go any further, you might be wondering what a gradient is?

Consider that you are holding a ball that is lying on the rim of a bowl. When you lose the ball, it travels in the steepest direction until it reaches the bowl's bottom. A gradient directs the ball in the steepest way possible to the local minimum, which is the bowl's bottom.

Gradient descent works with a set of coefficients, calculates their cost, and looks for a cost value that is lower than the current one. It shifts to a lesser weight and updates the values of the coefficients. The procedure continues until the local minimum is found. A local minimum is a point beyond which it is impossible to go any farther.

For the most part, gradient descent is the best option. It does, however, have significant drawbacks. Calculating the gradients is time-consuming when the data is large. For convex functions, gradient descent works well, but it doesn't know how far to travel down the gradient for nonconvex functions. Gradient descent works well for convex functions.

Stochastic Gradient Descent Deep Learning Optimizer-

On large datasets, gradient descent may not be the best solution. We use stochastic gradient descent to solve the problem. The word stochastic refers to the algorithm's underlying unpredictability. Instead of using the entire dataset for each iteration, we use a

random selection of data batches in stochastic gradient descent. As a result, we only sample a small portion of the dataset. The first step in this technique is to choose the starting parameters and learning rate. Then, in each iteration, mix the data at random to get an estimated minimum. When compared to the gradient descent approach, the path taken by the algorithm is full of noise since we are not using the entire dataset but only chunks of it for each iteration.

As a result, SGD requires more iterations to attain the local minimum. The overall computing time increases as the number of iterations increases. However, even when the number of iterations is increased, the computation cost remains lower than that of the gradient descent optimizer. As a result, if the data is large and the processing time is a consideration, stochastic gradient descent should be favored over batch gradient descent.

Mini-batch Stochastic Gradient Descent-

Mini batch SGD straddles the two preceding concepts, incorporating the best of both worlds. It takes training samples at random from the entire dataset (the so-called mini-batch) and computes gradients just from these. By sampling only a fraction of the data, it aims to approach Batch Gradient Descent.

We require fewer rounds because we're utilizing a chunk of data rather than the entire dataset. As a result, the mini-batch gradient descent technique outperforms both stochastic and batch gradient descent algorithms. This approach is more efficient and reliable than previous gradient descent variations. Because the method employs batching, all of the training data does not need to be placed into memory, making the process more efficient. In addition, the cost function in mini-batch gradient descent is noisier than that in batch gradient descent but smoother than that in stochastic gradient descent. Mini-batch gradient descent is therefore excellent and delivers a nice mix of speed and precision.
Mini-batch SGD is the most often utilized version in practice since it is both computationally inexpensive and produces more stable convergence.

Adagrad(Adaptive Gradient Descent) Optimizer -

Adagrad keeps a running total of the squares of the gradient in each dimension, and we adjust the learning rate depending on that total in each update. As a result, each parameter has a variable learning rate (or an adaptive learning rate). Furthermore, when we use the root of the squared gradients, we only consider the magnitude of the gradients, not the sign. We can observe that the learning rate is reduced when the gradient changes rapidly. The

learning rate will be higher when the gradient changes slowly. Due to the monotonic growth of the running squared sum, one of Adagrad's major flaws is that the learning rate decreases with time.

RMSprop (Root Mean Square) Optimizer-

Among deep learning aficionados, the RMS prop is a popular optimizer. This might be due to the fact that it hasn't been published but is nonetheless well-known in the community. RMS prop is a natural extension of RPPROP's work. The problem of fluctuating gradients is solved by RPPROP. The issue with the gradients is that some were modest while others may be rather large. As a result, establishing a single learning rate may not be the ideal option. RPPROP adjusts the step size for each weight based on the sign of the gradient. The two gradients are initially compared for signs in this technique.

Adam Deep Learning Optimizer-

To update network weights during training, this optimization approach is a further development of stochastic gradient descent. Unlike SGD, Adam optimizer modifies the learning rate for each network weight independently, rather than keeping a single learning rate for the entire training. The Adam optimizers inherit both Adagrad and RMS prop algorithm characteristics. Instead of using the first moment (mean) like in RMS Prop, Adam employs the second moment of the gradients to modify learning rates. We take the second instance of the gradients to imply the uncentered variance (we don't remove the mean).

AdaDelta Deep Learning Optimizer -

AdaDelta is a more powerful variant of the AdaGrad optimizer. It is based on adaptive learning and is intended to address the major shortcomings of AdaGrad and the RMS prop optimizer. The fundamental disadvantage of the two optimizers mentioned above is that the starting learning rate must be set manually. Another issue is the decreasing learning rate, which eventually becomes infinitesimally tiny. As a result, after a given number of iterations, the model can no longer acquire new information.

Normalization in Machine Learning

Normalization is a scaling technique in Machine Learning applied during data preparation to change the values of numeric columns in the dataset to use a common scale. It is not necessary for all datasets in a model. It is required only when features of machine learning models have different ranges.

Mathematically, we can calculate normalization with the below formula:

Xn = (X - Xminimum) / (Xmaximum - Xminimum)

- Xn = Value of Normalization
- Xmaximum = Maximum value of a feature
- Xminimum = Minimum value of a feature

Example: Let's assume we have a model dataset having maximum and minimum values of feature as mentioned above. To normalize the machine learning model, values are shifted and rescaled so their range can vary between 0 and 1. This technique is also known as **Min-Max scaling**. In this scaling technique, we will change the feature values as follows:

Case1- If the value of X is minimum, the value of Numerator will be 0; hence Normalization will also be 0.
Xn = (X - Xminimum) / (Xmaximum - Xminimum)
Put X =Xminimum in above formula, we get;
Xn = Xminimum- Xminimum/ (Xmaximum - Xminimum)
Xn = 0.

Case2- If the value of X is maximum, then the value of the numerator is equal to the denominator; hence Normalization will be 1.
Xn = (X - Xminimum) / (Xmaximum - Xminimum)
Put X =Xmaximum in above formula, we get;
Xn = Xmaximum - Xminimum/ (Xmaximum - Xminimum)
Xn = 1.

Case3- On the other hand, if the value of X is neither maximum nor minimum, then values of normalization will also be between 0 and 1.
Hence, Normalization can be defined as a scaling method where values are shifted and rescaled to maintain their ranges between 0 and 1, or in other words; it can be referred to as **Min-Max scaling technique**.

Normalization techniques in Machine Learning

Although there are so many feature normalization techniques in Machine Learning, few of them are most frequently used. These are as follows:

- **Min-Max Scaling:** This technique is also referred to as scaling. As we have already discussed above, the Min-Max scaling method helps the dataset to shift and rescale the values of their attributes, so they end up ranging between 0 and 1.

- **Standardization scaling:** Standardization scaling is also known as **Z-score** normalization, in which values are centered around the mean with a unit standard deviation, which means the attribute becomes zero and the resultant distribution has a unit standard deviation.

Mathematically, we can calculate the standardization by subtracting the feature value from the mean and dividing it by standard deviation.

Hence, standardization can be expressed as follows:

$$X' = \frac{X - \mu}{\sigma}$$

Here, **μ** represents the mean of feature value, and **σ** represents the standard deviation of feature values. However, unlike Min-Max scaling technique, feature values are not restricted to a specific range in the standardization technique. This technique is helpful for various machine learning algorithms that use distance measures such as **KNN, K-means clustering, and Principal component analysis**, etc. Further, it is also important that the model is built on assumptions and data is normally distributed.

References

1. https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/